```
import { useState } from "react";

// ─── Mode config ──────────────────────────────────────────────────────────────
// chat    = Claude chat session (writing, specs, content)
// cowork  = Cowork desktop app with Claude (file ops, execution)
// code    = You write code (Claude Code / Cowork / direct)
// offline = No Claude needed (Midjourney, designer, chasing people)
const modeConfig = {
  chat:    { label: "Chat",    color: "bg-indigo-950 text-indigo-300 border borde
  cowork:  { label: "Cowork",  color: "bg-violet-950 text-violet-300 border borde
  code:    { label: "Code",    color: "bg-cyan-950 text-cyan-300 border border-cy
  offline: { label: "Offline", color: "bg-orange-950 text-orange-300 border borde
};

// ─── Task data ────────────────────────────────────────────────────────────────
// done       → Oso has completed their side — tell Claude to verify
// today      → Suggested for tonight/tomorrow — flagged in plan panel
// mode       → How this task gets done (chat/cowork/code/offline)
// offline    → Can be done without internet or Claude
// dependsOn  → Task IDs that must be done first

const tasks = [

  // ──────────────────────────────────────────────────────────────────────────
  // PHASE 01–05 — FOUNDATION, VOICE, DESIGN SYSTEM, ONBOARDING, THEMING
  // All complete. The world is defined.
  // ──────────────────────────────────────────────────────────────────────────

  {
    phase: "01 — Brand Foundation",
    status: "complete",
    estimate: "Complete ✓",
    items: [
      { id: 1,  task: "Brand vision, mission and core value proposition",   statu
      { id: 2,  task: "Target audience and subculture strategy",            statu
      { id: 3,  task: "Brand line",                                         statu
      { id: 4,  task: "Brand positioning statement",                        statu
      { id: 5,  task: "Updated PRD v3",                                     statu
    ]
  },
  {
    phase: "02 — Brand Voice & Avatar Content",
    status: "inprogress",
    estimate: "~2–4h chat remaining — #18 and #19 are chatbot-prep, not blocking
```

```
  items: [
    { id: 6,  task: "Brand voice personality and principles",             statu
    { id: 7,  task: "Voice by surface",                                   statu
    { id: 8,  task: "Avatar naming",                                      statu
    { id: 9,  task: "Avatar personas — appearance, lifestyle, aesthetic", statu
    { id: 10, task: "Avatar voice profiles and sample lines",              stat
    { id: 11, task: "Avatar introduction copy for onboarding",            statu
    { id: 12, task: "Avatar first words on companion confirmation",       stat
    { id: 13, task: "Avatar return session greetings",                    statu
    { id: 14, task: "Avatar resonance lines for profile card appearances", stat
    { id: 15, task: "Extended character backstories — all four avatars",  statu
    { id: 16, task: "Avatar character references locked",                 stat
    { id: 17, task: "Avatar naming language in-app",                      statu
    { id: 136, task: "Per-avatar one-liners — all 22 Major Arcana",       statu
    { id: 137, task: "Avatar LLM seed prompts — all four avatars",        statu
    { id: 18, task: "Avatar character profile expansion — deeper lore and dialo
    { id: 19, task: "Avatar coach/mentor voice scripts — tarot scenario library
  ]
},
{
  phase: "03 — Product Design System",
  status: "complete",
  estimate: "Complete ✓",
  items: [
    { id: 20, task: "Visual language direction",                          stat
    { id: 21, task: "Colour system — full hex values confirmed",          stat
    { id: 22, task: "Composition system and card framing rules",          stat
    { id: 23, task: "Typography — fonts selected and scale locked",       stat
    { id: 24, task: "Motion system and card state model",                 stat
    { id: 25, task: "Elemental motion skins",                             stat
    { id: 26, task: "Avatar visual rule sheet",                           stat
    { id: 27, task: "Anime theme component variations",                   stat
    { id: 28, task: "Avatar UI theme mapping locked",                     stat
    { id: 29, task: "Component template",                                 stat
    { id: 30, task: "Aura treatment rules — all card contexts",           stat
    { id: 31, task: "Avatar appearance in all app states",                stat
  ]
},
{
  phase: "04 — Onboarding",
  status: "complete",
  estimate: "Complete ✓",
  items: [
    { id: 32, task: "Three-phase onboarding structure",                   stat
    { id: 33, task: "Command-line terminal entry screens",               stat
    { id: 34, task: "Birth card two-card system",                         stat
    { id: 35, task: "Majestic Profile reveal screens",                    stat
```

```
    { id: 36, task: "Profile summary — first codex entry",                  stat
    { id: 37, task: "World quiz — four external scenario questions",        sta
    { id: 38, task: "Avatar recommendation screen",                          stat
    { id: 39, task: "Companion confirmation and UI theme unlock",            sta
    { id: 40, task: "First draw — ritual established",                       stat
    { id: 41, task: "After onboarding flows",                                stat
    { id: 42, task: "Full onboarding v2 document",                          stat
    { id: 43, task: "Birth card calculation technical spec",                 sta
    { id: 44, task: "Personality and soul card interpretation copy — all 22 Maj
  ]
},
{
  phase: "05 — Theming",
  status: "complete",
  estimate: "Complete ✓",
  items: [
    { id: 45, task: "Theme direction confirmed — four avatar themes",       sta
    { id: 46, task: "Threshold City theme spec — base world",               stat
    { id: 47, task: "Avatar accent system — all four",                      stat
    { id: 48, task: "Folklore Signal theme spec — Destiny / Water",         stat
    { id: 49, task: "Codex Arc theme spec — Eli / Air",                     statu
    { id: 50, task: "Magical Relic theme spec — Olivia / Earth",            stat
    { id: 51, task: "Theme transition rules — switching between avatars",   sta
    { id: 52, task: "Shared elements across all themes",                    sta
  ]
},

// ──────────────────────────────────────────────────────────────────────────
// PHASE 06–08 — VISUAL PRODUCTION, CARD DESIGN, SURFACE SPECS
// Asset production + content + all surface specs. Unblocks build phase.
// ──────────────────────────────────────────────────────────────────────────

{
  phase: "06 — Avatar & Visual Production",
  status: "inprogress",
  estimate: "Asset production phase. Emblems done. Lottie + Pixel Elder art sti
  items: [
    { id: 53, task: "AI prompt library for avatar generation",              stat
    { id: 54, task: "Casper — all three states generated and approved",     stat
    { id: 55, task: "Olivia — all three states generated and approved",     stat
    { id: 56, task: "Olivia — reflective state",                            stat
    { id: 57, task: "Destiny — active and reflective states generated",     stat
    { id: 58, task: "Destiny — neutral state",                              stat
    { id: 59, task: "Eli — all three states generated and approved",        stat
    { id: 60, task: "Pixel Elder character spec",                           stat
    { id: 61, task: "Pixel Elder — pixel art production",                   stat
    { id: 62, task: "Avatar emblem set — design production",                stat
```

```
    { id: 63, task: "Avatar emblem set — SVG masters and PNG exports",     stat
    { id: 64, task: "Avatar emblem — Lottie animations (trace draw-on + pulse)"
    { id: 65, task: "Avatar motion rules",                                  sta
    { id: 66, task: "Aura layer tested on mockups",                        stat
    { id: 67, task: "Avatar appearance tested in portal at all three scales", s
    { id: 68, task: "Card frame system",                                   stat
    { id: 69, task: "Final typography selections",                         stat
    { id: 70, task: "Colour confirmation and swatch system",               stat
    { id: 71, task: "Emblem and glyph system — spec",                      statu
    { id: 72, task: "Visual identity brief — full handoff document",       stat
    { id: 73, task: "Brand guidelines document",                           stat
    { id: 74, task: "World glyph set — elemental seals production",        stat
    { id: 75, task: "World glyph set — signal markers production",         stat
    { id: 76, task: "Major Arcana sigils — 22 card sigils",               statu
    { id: 77, task: "Card frame — final visual production",                stat
    { id: 78, task: "App icon production",                                 stat
    { id: 79, task: "Script font — WOFF2 file",                           statu
  ]
},
{
  phase: "07 — Card Design & Content",
  status: "inprogress",
  estimate: "~4h chat (4 prompt libraries) + Midjourney generation time (78 car
  items: [
    { id: 80, task: "AI prompt library — Major Arcana",                   stat
    { id: 81, task: "AI prompt library — Wands (Fire / Casper)",          statu
    { id: 82, task: "AI prompt library — Cups (Water / Destiny)",         statu
    { id: 83, task: "AI prompt library — Swords (Air / Eli)",             statu
    { id: 84, task: "AI prompt library — Pentacles (Earth / Olivia)",     statu
    { id: 85, task: "Major Arcana generation — 22 cards",                 statu
    { id: 86, task: "Minor Arcana generation — Wands: 14 cards",          statu
    { id: 87, task: "Minor Arcana generation — Cups: 14 cards",           statu
    { id: 88, task: "Minor Arcana generation — Swords: 14 cards",         statu
    { id: 89, task: "Minor Arcana generation — Pentacles: 14 cards",      statu
    { id: 90, task: "Card back design",                                   stat
    { id: 91, task: "Card title and metadata treatment",                  stat
    { id: 92, task: "Aura category mapping — all 78 cards",               statu
    { id: 93, task: "Accessibility copy rules for cards",                 stat
  ]
},
{
  phase: "08 — Core Product Surfaces",
  status: "inprogress",
  estimate: "Mostly complete. Daily draw component spec still needs full animat
  items: [
    { id: 121, task: "Daily draw — component spec",                        stat
    { id: 122, task: "Daily draw — meditation/reflection ritual spec",     stat
```

```
    { id: 123, task: "Reading screen — 1-card spread spec",                stat
    { id: 124, task: "Reading screen — 3-card spread spec",                stat
    { id: 125, task: "Reading initiation — spread selection UI",           stat
    { id: 126, task: "Deck browser and codex — component spec",           statu
    { id: 127, task: "Journal and reflection surface — spec",              stat
    { id: 128, task: "Majestic Profile surface — spec",                    stat
    { id: 129, task: "Notification and reminder language",                 stat
    { id: 130, task: "Empty states and loading states",                    stat
    { id: 131, task: "Avatar switching — full UI flow",                    stat
    { id: 154, task: "Altar & ritual spec — avatar altars, talismans and breath
    { id: 157, task: "Quiz skip UX — onboarding companion selection without qui
    { id: 158, task: "Sound design spec — reading audio events",           stat
    { id: 159, task: "Subscription tier spec — gating logic and UX",      statu
    { id: 160, task: "Supabase schema — subscription fields",              stat
    { id: 161, task: "Subscription screen UI",                            stat
    { id: 162, task: "RevenueCat webhook — Supabase edge function",       stat
    { id: 163, task: "Auth prompt — Screen 11 integration",               stat
    { id: 164, task: "Auth & monetisation positioning spec",               sta
    { id: 155, task: "Altar world-names — Threshold City lore names for altar o
    { id: 156, task: "Sound design — ambient audio for card readings",      sta
    { id: 169, task: "Navigation architecture spec — three-state spatial system
    { id: 170, task: "Ground state / Command center — visual spec",
    { id: 171, task: "Home screen content system spec — Co-Star + pop culture r
    { id: 172, task: "Portal visual treatment — define reference, mood, mechani
    { id: 173, task: "Self state desk scene — Lottie animation spec",
    { id: 174, task: "Self state environments — 4 avatar-specific desk scenes",
    { id: 175, task: "Pop culture reference database — Major Arcana × 4 avatars
    { id: 176, task: "Avatar colour temperature per nav state — implement",
    { id: 177, task: "Portal transition animation — production",
  ]
},

// ──────────────────────────────────────────────────────────────────────
// PHASE 09–10 — INFRASTRUCTURE + BUILD
// The actual construction. Schema first, then everything else.
// ──────────────────────────────────────────────────────────────────────

{
  phase: "09 — Infrastructure & Security",
  status: "inprogress",
  estimate: "~2–4h cowork/code. Schema + RLS is the critical unblock for build
  items: [
    { id: 98,  task: "Tech stack confirmation",                            sta
    { id: 132, task: "App scaffold and project structure",                 stat
    { id: 112, task: "Auth flow — Supabase Auth implementation",           stat
    { id: 111, task: "Database schema — provision in Supabase dashboard", statu
    { id: 145, task: "Security — RLS policies + API key protection",       stat
```

```
    { id: 150, task: "App Store accounts — Apple Developer + Google Play", stat
    { id: 146, task: "Error monitoring — Sentry integration",              stat
    { id: 147, task: "Analytics — PostHog integration",                    stat
    { id: 153, task: "Deep linking — notification routing",                stat
  ]
},
{
  phase: "10 — Build",
  status: "inprogress",
  estimate: "~8–12 weeks. Portal + aura systems are the animation critical path
  items: [
    { id: 133, task: "Zustand state stores — authStore, avatarStore, profileSto
    { id: 134, task: "Major Arcana card data — TypeScript implementation", stat
    { id: 99,  task: "Birth card calculator — implement and test",          sta
    { id: 100, task: "Colour token file — implement in codebase",          stat
    { id: 101, task: "Typography implementation",                          sta
    { id: 102, task: "Avatar accent theme switcher — implement",           stat
    { id: 109, task: "Portal system — implement (Arch + Living Circle)",   stat
    { id: 108, task: "Aura state system — implement",                      stat
    { id: 106, task: "Living signal layer — implement",                    stat
    { id: 103, task: "Onboarding flow — build Phase 1 (Signal & Entry)",  statu
    { id: 104, task: "Onboarding flow — build Phase 2 (Majestic Profile)", stat
    { id: 105, task: "Onboarding flow — build Phase 3 (Choose Companion)", stat
    { id: 107, task: "Card flip and reveal animation",                     stat
    { id: 110, task: "Pixel Elder — integration",                          stat
    { id: 113, task: "Offline cache — implement",                          stat
    { id: 114, task: "Push notification system — implement",               stat
    { id: 115, task: "Payment and subscription — RevenueCat",              stat
  ]
},

// ——————————————————————————————————————————————————————————————
// PHASE 11–12 — PRE-LAUNCH + GO TO MARKET
// Everything needed to actually ship. Legal, compliance, store presence.
// ——————————————————————————————————————————————————————————————

{
  phase: "11 — Pre-Launch",
  status: "todo",
  estimate: "~3–4h chat (privacy policy, compliance check) + ~1h code (TestFlig
  items: [
    { id: 152, task: "TestFlight + Android beta track setup",              stat
    { id: 148, task: "Privacy policy + terms of service",                  stat
    { id: 149, task: "GDPR — account deletion flow",                       stat
    { id: 151, task: "App Store review — pre-submission compliance check", stat
  ]
},
```

```
  {
    phase: "12 — Go To Market",
    status: "todo",
    estimate: "~3–4h chat (copy: app store + landing + social) + visual productio
    items: [
      { id: 116, task: "App store copy — title, subtitle, description",      stat
      { id: 117, task: "App store visuals — screenshots and preview",        stat
      { id: 118, task: "Landing page copy and structure",                   stat
      { id: 119, task: "Social presence strategy",                          stat
      { id: 120, task: "Launch narrative — soft launch and community seeding", st
    ]
  },
  {
    phase: "13 — Future Releases",
    status: "future",
    estimate: "Post-v1 and post-beta scope. Ideas and enhancements captured here
    items: [
      { id: 138, task: "Journal moodboard mode — canvas editor",            stat
      { id: 139, task: "Alternative card decks",                             sta
      { id: 140, task: "Full environmental variation per avatar theme",       st
      { id: 141, task: "AR card reveal feature",                              st
      { id: 142, task: "Seasonal and time-based world shifts",                st
      { id: 143, task: "Pattern tracking and insight summaries — Journal",    st
      { id: 144, task: "Social and sharing features",                         st
    ]
  }
];

// ─── Config ──────────────────────────────────────────────────────────────────
const statusConfig = {
  complete:   { label: "Complete",     color: "bg-emerald-900 text-emerald-300 bo
  inprogress: { label: "In Progress",  color: "bg-amber-900 text-amber-300 border
  todo:       { label: "To Do",        color: "bg-slate-800 text-slate-400 border
  future:     { label: "Future",       color: "bg-purple-900 text-purple-300 bord
  cowork:     { label: "Cowork",       color: "bg-blue-900 text-blue-300 border b
  code:       { label: "Code",         color: "bg-cyan-900 text-cyan-300 border b
  offline:    { label: "Offline",      color: "bg-slate-700 text-slate-300 border
};

const phaseStatusConfig = {
  complete:   { bg: "bg-[#0d0d1a]", border: "border-emerald-900", heading: "text-
  inprogress: { bg: "bg-[#0d0d1a]", border: "border-amber-900",   heading: "text-
  todo:       { bg: "bg-[#0d0d1a]", border: "border-slate-700",   heading: "text-
  future:     { bg: "bg-[#0d0d1a]", border: "border-purple-800",  heading: "text-
  cowork:     { bg: "bg-[#0d0d1a]", border: "border-blue-800",    heading: "text-
  code:       { bg: "bg-[#0d0d1a]", border: "border-cyan-800",    heading: "text-
};
```

```
// ─── Component ──────────────────────────────────────────────────────────
export default function MajesticTracker() {
  const [filter, setFilter] = useState("all");
  const [expanded, setExpanded] = useState(() => {
    const init = {};
    tasks.forEach((_, i) => { init[i] = false; });
    tasks.forEach((p, i) => { if (p.status === "inprogress") init[i] = true; });
    return init;
  });
  const [planOpen, setPlanOpen] = useState(true);
  const [doneState, setDoneState] = useState(() => {
    const init = {};
    tasks.flatMap(p => p.items).forEach(item => { init[item.id] = item.done; });
    return init;
  });

  const toggleDone = (id, e) => {
    e.stopPropagation();
    setDoneState(prev => ({ ...prev, [id]: !prev[id] }));
  };

  const allItems = tasks.flatMap(p => p.items);
  const totalTasks   = allItems.length;
  const completedTasks  = allItems.filter(i => i.status === "complete").length;
  const inProgressTasks = allItems.filter(i => i.status === "inprogress").length;
  const todoTasks       = allItems.filter(i => i.status === "todo").length;
  const progress = Math.round((completedTasks / totalTasks) * 100);

  const todayItems   = allItems.filter(i => i.today);
  const tonightItems = todayItems.filter(i => i.offline);
  const chatItems    = todayItems.filter(i => i.mode === "chat");
  const coworkItems  = todayItems.filter(i => i.mode === "cowork");

  const taskMap = {};
  allItems.forEach(item => { taskMap[item.id] = item.task; });

  const filteredTasks = tasks.map(phase => ({
    ...phase,
    items: (() => {
      if (filter === "today")   return phase.items.filter(i => i.today);
      if (filter === "offline") return phase.items.filter(i => i.offline && i.sta
      if (filter !== "all")     return phase.items.filter(i => i.status === filte
      return phase.items;
    })()
  })).filter(phase => phase.items.length > 0);
```

```
const flags = [
  { text: "BLOCKER: Portal visual treatment (#172) undefined — Luke must provid
  { text: "DECISION NEEDED: Codex expansion eligibility trigger — subscription,
  { text: "DECISION NEEDED: Avatar colour temperature at ground state — city ne
  { text: "DECISION NEEDED: Daily draw method — algorithmic, user-selected, or
  { text: "DECISION NEEDED: Drink pour animation — universal or avatar-specific
  { text: "Sound design spec (#158) needed before audio assets can be sourced —
  { text: "Dig Deeper spec updated to v2.0 (majestic-dig-deeper-spec-v2.md) — u
  { text: "Subscription schema additions (#160) needed before RevenueCat build
  { text: "New specs created this session — upload to project knowledge: majest
  { text: "Luke's font confirmed: Remachine Script at app/assets/fonts/Remachin
];

return (
  <div className="min-h-screen font-sans" style={{ background: "#090910" }}>

    {/* ── Header ── */}
    <div style={{ background: "linear-gradient(160deg, #0f0f1f 0%, #1a1a2e 60%,
      <div className="max-w-4xl mx-auto">
        <p className="text-purple-400 text-xs tracking-[0.25em] uppercase mb-1
        <h1 className="text-4xl font-bold tracking-tight mb-1" style={{ color:
        <p className="text-purple-400 italic text-sm mb-8" style={{ fontFamily:

        {/* Progress bar */}
        <div className="mb-2 flex justify-between text-xs text-slate-400 font-m
          <span>{completedTasks} of {totalTasks} tasks complete</span>
          <span style={{ color: "#9500FF" }}>{progress}%</span>
        </div>
        <div className="w-full rounded-full h-1.5 mb-5" style={{ background: "#
          <div className="h-1.5 rounded-full transition-all duration-700" style
        </div>
        <div className="flex gap-6 flex-wrap mb-8">
          {[["bg-emerald-400", completedTasks + " complete"], ["bg-amber-400",
            <div key={label} className="flex items-center gap-2">
              <div className={"w-1.5 h-1.5 rounded-full " + dot} />
              <span className="text-xs text-slate-400 font-mono">{label}</span>
            </div>
          ))}
        </div>

        {/* ── Tonight & Tomorrow Plan ── */}
        <div className="mb-6 rounded-lg border border-yellow-900 overflow-hidde
          <button onClick={() => setPlanOpen(p => !p)} className="w-full px-4 p
            <span className="text-xs font-mono font-bold tracking-widest upperc
            <span className="text-yellow-700 text-xs">{planOpen ? "▲" : "▼"}</s
          </button>
          {planOpen && (
```

```
    <div className="px-4 pb-4 space-y-4 border-t border-yellow-900">
      {tonightItems.length > 0 && (
        <div className="mt-3">
          <p className="text-xs font-mono text-orange-400 tracking-wide
          <div className="space-y-1">
            {tonightItems.map(item => (
              <div key={item.id} className="flex items-start gap-2 text
                <span className="flex-shrink-0 mt-0.5 text-orange-500">
                <span><span className="text-orange-600 mr-1">#{item.id}
              </div>
            ))}
          </div>
        </div>
      )}
      {chatItems.length > 0 && (
        <div>
          <p className="text-xs font-mono text-indigo-400 tracking-wide
          <div className="space-y-1">
            {chatItems.map(item => (
              <div key={item.id} className="flex items-start gap-2 text
                <span className="flex-shrink-0 mt-0.5 text-indigo-500">
                <span><span className="text-indigo-600 mr-1">#{item.id}
              </div>
            ))}
          </div>
        </div>
      )}
      {coworkItems.length > 0 && (
        <div>
          <p className="text-xs font-mono text-violet-400 tracking-wide
          <div className="space-y-1">
            {coworkItems.map(item => (
              <div key={item.id} className="flex items-start gap-2 text
                <span className="flex-shrink-0 mt-0.5 text-violet-500">
                <span><span className="text-violet-600 mr-1">#{item.id}
              </div>
            ))}
          </div>
        </div>
      )}
    </div>
  )}
</div>

{/* Flags */}
<div className="space-y-2">
  <p className="text-xs text-slate-500 font-mono tracking-widest upperc
```

```
      {flags.map((f, i) => (
        <div key={i} className={`flex items-start gap-2 px-3 py-2 rounded t
          <span className="mt-0.5 flex-shrink-0">{f.severity === "amber" ?
          <span>{f.text}</span>
        </div>
      ))}
    </div>

    {/* Mode legend */}
    <div className="mt-5 flex gap-3 flex-wrap">
      <p className="text-xs text-slate-600 font-mono self-center">Mode:</p>
      {Object.entries(modeConfig).map(([key, cfg]) => (
        <span key={key} className={"text-xs px-2 py-0.5 rounded font-mono f
      ))}
      <span className="text-xs px-2 py-0.5 rounded font-mono font-medium bo
      <span className="text-xs px-2 py-0.5 rounded font-mono font-medium bo
    </div>
  </div>
</div>

{/* ── Filter bar ── */}
<div className="sticky top-0 z-10 border-b border-slate-800" style={{ backg
  <div className="max-w-4xl mx-auto px-6 py-3 flex gap-2 flex-wrap">
    {[
      ["all", "All"],
      ["today", "⭐ Tonight/Tomorrow"],
      ["offline", "⚡ Offline"],
      ["inprogress", "In Progress"],
      ["todo", "To Do"],
      ["complete", "Complete"],
    ].map(([f, label]) => (
      <button key={f} onClick={() => setFilter(f)}
        className={"px-3 py-1.5 rounded text-xs font-mono font-medium trans
        style={filter === f ? { background: f === "today" ? "#7a6000" : f =
        {label}
      </button>
    ))}
  </div>
</div>

{/* ── Phase list ── */}
<div className="max-w-4xl mx-auto px-6 py-6 space-y-3">
  {filteredTasks.map((phase, phaseIndex) => {
    const originalIndex = tasks.findIndex(t => t.phase === phase.phase);
    const cfg = phaseStatusConfig[phase.status] || phaseStatusConfig["todo"
    const isOpen = expanded[originalIndex];
    const phaseComplete = phase.items.filter(i => i.status === "complete").
```

```
return (
  <div key={phaseIndex} className={"rounded-lg border overflow-hidden "
    <button onClick={() => setExpanded(prev => ({ ...prev, [originalInd
      className="w-full px-5 py-4 flex items-center justify-between hov
      <div className="flex-1 min-w-0">
        <div className="flex items-center gap-3 flex-wrap mb-1">
          <span className={"text-sm font-bold font-mono " + cfg.heading
          <span className={"text-xs px-2 py-0.5 rounded font-mono font-
            {statusConfig[phase.status].label}
          </span>
        </div>
        <p className="text-xs text-slate-600 font-mono">{phase.estimate
      </div>
      <div className="flex items-center gap-3 ml-3 flex-shrink-0">
        <span className="text-xs text-slate-500 font-mono">{phaseComple
        <span className="text-slate-600 text-xs">{isOpen ? "▲" : "▼"}</
      </div>
    </button>

    {isOpen && (
      <div className="px-5 pb-4 space-y-1.5">
        {phase.items.map(item => {
          const s = statusConfig[item.status] || statusConfig["todo"];
          const m = modeConfig[item.mode];
          const isChecked = doneState[item.id];
          const incompleteDeps = item.dependsOn.filter(depId => {
            const dep = allItems.find(i => i.id === depId);
            return dep && dep.status !== "complete";
          });
          const completeDeps = item.dependsOn.filter(depId => {
            const dep = allItems.find(i => i.id === depId);
            return dep && dep.status === "complete";
          });

          return (
            <div key={item.id} className={"flex items-start gap-3 p-3 r

              {/* Checkbox */}
              <button
                onClick={(e) => toggleDone(item.id, e)}
                className={"flex-shrink-0 mt-0.5 w-4 h-4 rounded border
                title={isChecked ? "Mark as not done" : "Mark as done —
              >
                {isChecked && <span className="text-white text-xs leadi
              </button>

              <div className="flex-1 min-w-0">
```

```
                    {/* Top row: task name + badges */}
                    <div className="flex items-start justify-between gap-2
                      <p className={"text-sm font-medium flex-1 " + (item.s
                        <span className="text-slate-600 text-xs font-mono m
                        {item.task}
                      </p>
                      <div className="flex items-center gap-1.5 flex-shrink
                        {item.today && (
                          <span className="text-xs px-1.5 py-0.5 rounded fo
                        )}
                        <span className={"text-xs px-1.5 py-0.5 rounded fon
                        {m && <span className={"text-xs px-1.5 py-0.5 round
                        {item.offline && item.status !== "complete" && (
                          <span className="text-xs px-1.5 py-0.5 rounded fo
                        )}
                      </div>
                    </div>

                    {/* Note */}
                    {item.note && <p className="text-xs text-slate-500 mt-1

                    {/* Dependencies */}
                    {item.status !== "complete" && (incompleteDeps.length >
                      <div className="mt-1.5 flex flex-wrap gap-1">
                        {incompleteDeps.map(depId => (
                          <span key={depId} className="text-xs font-mono px
                            ↳ needs #{depId} {taskMap[depId] ? `— ${taskMap
                          </span>
                        ))}
                        {completeDeps.map(depId => (
                          <span key={depId} className="text-xs font-mono px
                            ✓ #{depId} done
                          </span>
                        ))}
                      </div>
                    )}
                  </div>
                </div>
              );
            })}
          </div>
        )}
      </div>
    );
  })}
</div>
```

```
      <div className="max-w-4xl mx-auto px-6 py-8 text-center">
        <p className="text-xs font-mono" style={{ color: "#9500FF" }}>You trusted
      </div>
    </div>
  );
}
```